

Blatt 07: Generics, Sealed Types, Stream-API, Logging

WICHTIG:

CODE IST AUF MASTER BRANCH!!!

Aufgabe Zoo

Sealed Interfaces (versiegelte Schnittstellen) erlauben es Ihnen, strikt zu kontrollieren, welche Klassen oder Interfaces sie implementieren bzw. erweitern dürfen.

Traditionell war die Implementierung eines Interfaces offen: Jede beliebige Klasse in Ihrem Projekt konnte das Interface implementieren. Bei *Sealed Interfaces* ist das anders.

- Sie bestimmen explizit, wer das Interface implementieren darf. Andere Klassen werden vom Compiler abgewiesen
- **Das Schlüsselwort permits (Java):** Der Compiler weiß zur Kompilierzeit genau, welche Ausprägungen des Interfaces existieren
- Da der Compiler alle erlaubten Unterklassen oder Implementierungen kennt, können Sie (besonders bei switch-Ausdrücken) auf den default-Zweig verzichten. Der Compiler warnt Sie, falls Sie einen Fall vergessen haben
- Wenn Sie eine Bibliothek schreiben, können Sie verhindern, dass Dritte Ihr Interface unerlaubt erweitern, und so die Architektur Ihrer Software schützen

Record-Klassen sind spezielle Klassen zur einfachen Speicherung unveränderlicher Daten. Sie dienen primär dazu, sogenannten „Boilerplate-Code“ (wie Konstruktoren, Getter, equals(), hashCode() und toString()) drastisch zu reduzieren, der bei herkömmlichen Klassen manuell geschrieben werden muss.

- **Eigene Methoden:** Sie können innerhalb eines Records beliebige eigene (auch statische) Methoden definieren
- **Keine weiteren Instanzvariablen:** Ein Record kann keine weiteren Objekt-Attribute besitzen als die in den runden Klammern definierten
- **Keine Vererbung:** Records sind implizit final und können keine anderen Klassen erweitern (aber sie können Interfaces implementieren)

Ein traditionelles class-Datenobjekt erfordert viel Schreibaufwand. Mit einem record sieht das so aus:

```
public record Person(String vorname, String nachname, int alter) {}
```

Daraus generiert Java unter der Haube automatisch:

- Einen Konstruktor `public Person(String vorname, String nachname, int alter)`
- Die Getter-Methoden `vorname()`, `nachname()` und `alter()`
- Die Standardmethoden `equals()`, `hashCode()` und `toString()`

Generische Klassen

Eine generische Klasse ist ein Bauplan, der Platzhalter für Datentypen verwendet. Der eigentliche Datentyp wird erst bei der Erstellung des Objekts (der Instanziierung) festgelegt. Dies ermöglicht es, flexiblen und wiederverwendbaren Code zu schreiben.

// Definition der generischen Klasse mit dem Platzhalter <T>

public class Box<T>

Box<String> stringBox = new Box<>(); (Die Box nimmt jetzt nur Zeichenketten auf)

Box<Integer> intBox = new Box<>(); (Die Box nimmt jetzt nur ganze Zahlen auf)

Kein Tier kann in einem Gehege mehrfach vorkommen. Lösen Sie dies durch die Auswahl einer geeigneten inneren Datenstruktur und begründen Sie Ihre Wahl kurz.

LinkedHashSet<T>, da Set Duplikate verhindert und eine Einfügreihenfolge erhält.

Wählen Sie die Parameter und Rückgabetyphen geeignet und begründen Sie Ihre Wahl kurz.

Listen (List<T>)

- kompatibel mit Streams
- gut iterierbar
- z. B. List<Mammal>

Map<Class<?>, Long>

- perfekte Struktur für Statistiken
- Class = Tierart
- Long = Anzahl

String

- kompakt und universell

Enclosure<? Extends Animal>

- erlaubt unterschiedliche Tierarten
- verhindert falsche Kombinationen

Aufgabe 3: Reflektion

Beantworten Sie die folgenden Fragen kurz und prägnant (Stichpunkte sind ausreichend). Geben Sie Ihre Antworten zusammen mit Ihrem Code in Ihrem Repo ab (z.B. im README.md).

1. Generics

Wo helfen Ihnen die Generics im Zoo-Szenario, Fehler bereits zur Compile-Zeit zu vermeiden?

- Verhindern, dass ein Gehege Tiere falscher Art enthält
- Typgrenzen (T extends Animal) verhindern, dass Nicht-Tiere in Gehege gelangen
- Spezialisierte Gehege (Aquarium extends Enclosure<Fish>) sind statisch sicher
- Der Compiler verhindert falsche Kombinationen bevor der Code ausgeführt wird

Nennen Sie ein Beispiel aus Ihrer Implementierung, bei dem falsche Tier-Gehege-Kombinationen durch den Typchecker verhindert werden.

- MammalHouse extends Enclosure<Mammal>
 - verhindert, dass z. B. ein Sunfish (Fisch) hinzugefügt wird
 - mammalHouse.add(new Sunfish("Mond")); führt zu Compile-Fehler
- CatHouse extends Enclosure<Lykoi>
 - nur Lykoi erlaubt
 - catHouse.add(new Shoebill("Dumbo")); ist Compile-Fehler

2. Logging

Warum ist systematisches Logging mit einem Logger und Log-Leveln für ein Zoo-Management-System sinnvoller als Ausgaben mit IO.println?

- Es gibt ein Log-Level, welches die Wichtigkeit einer Software-Meldung klassifiziert
- Logs können gefiltert, umgeleitet, formatiert werden
- println ist immer gleich wichtig, also hat keine Priorisierung
- Logger kann in Produktion auf Datei, Konsole, Netzwerk schreiben
- Ermöglicht Debugging, Monitoring, Fehleranalyse
- println ist nicht abschaltbar, dafür Logger schon

In welchen Situationen würden Sie in Ihrem System die Log-Level INFO, WARNING und ggf. SEVERE verwenden?

INFO

- *Start einer Methode*
- *normale Aktionen: Tiere holen, Gehege hinzufügen*
- *Zusammenfassungen, Statusmeldungen*

WARNING

- *Gehege nicht gefunden*
- *Tier nicht gefunden*
- *ungültige Eingaben, aber System läuft weiter*

SEVERE

- *kritische Fehler*
- *Inkonsistenzen (z. B. null-Gehege)*
- *Zoo-Daten beschädigt*

3. Streams

Wo haben Ihnen Streams im Vergleich zu klassischen Schleifen beim Formulieren Ihrer Abfragen geholfen? Wo wurde es eher unübersichtlich?

- Kürzere, klarere Abfragen
- flatMap für „alle Tiere aus allen Gehegen“
- filter für Tierarten (z. B. nur Mammals)
- map für Typkonvertierung
- collect(groupingBy) für Typ-Zählung
- Bei Wildcards (? extends Animal)
- Bei groupingBy mit Generics